What is Version Control?

Version Control is a system used to **record and manage changes made to files over time**. It keeps a history of different versions of a project, so we can **see what changes were made, compare versions, and return to an earlier version when needed**.

In simple words, **version control helps us keep track of the different versions of a project without creating many separate copies of the files**.

It is especially useful when **multiple people work on the same project**, because it helps manage their changes and prevents one person's work from accidentally replacing another person's work.

Example:
Suppose you are working on a Python data science project. You make changes to your program today, more changes tomorrow, and later discover that the old program worked better. With version control, you can **view the previous version and restore it** if required.

Git is a commonly used example of a version control system

Why is Version Control Needed?

Version control is needed to **manage and keep track of changes made to a project over time**. It is useful for both individual work and team projects.

It is needed to:

- **Keep a history of changes** – We can see what changes were made, when they were made, and who made them.
- **Go back to an earlier version** – If a new change causes a problem, we can return to a previous working version.
- **Prevent loss of work** – The project history can be stored and recovered when files are accidentally changed or lost.
- **Support teamwork** – Multiple people can work on the same project, share their changes, and combine their work.
- **Find and fix mistakes** – By checking previous changes, it becomes easier to identify when a problem was introduced and correct it.

In simple words: Version control is needed to **save the history of a project, protect our work, undo mistakes, and make teamwork easier**.

Installing and Configuring Git

Git is a version control tool used to track and manage changes in project files. Before using Git, it needs to be installed on the computer and configured with basic user information.

1. Download Git

Download Git from the official Git website:

Download Git — Official Website

Git is available for **Windows, macOS, and Linux**.

2. Install Git

For Windows:

1. Download the Git installer.
2. Run the downloaded installer.
3. Follow the installation steps.
4. The default options are generally suitable for beginners.
5. After installation, open **Git Bash** or **Command Prompt**.

3. Check Git Installation

Type the following command:

```
git --version
```

If Git is installed correctly, it will display the installed Git version.

4. Configure Git

After installing Git, set your **name** and **email address**. This information is associated with the commits you make.

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

The --global option applies these settings to all Git repositories for your user account on that computer. Git

5. Check the Configuration

To check your Git settings, use:

```
git config --list
```

You can also check your name and email separately:

```
git config --global user.name
```

```
git config --global user.email
```

In simple words: Installing Git makes Git available on the computer, while configuring Git tells it **who is making the changes**.

Basic Git Commands

Git provides commands to create repositories, copy existing repositories, track changes, save changes, check the current state, and view project history. The following are some of the most basic commands.

1. git init

Purpose: Creates a new Git repository in the current folder. It creates the .git directory where Git stores the repository information.

Syntax:

```
git init
```

Example:

```
cd my-project
git init
```

2. git clone

Purpose: Creates a local copy of an existing Git repository, usually from a remote location such as GitHub.

Syntax:

```
git clone <repository-url>
```

Example:

```
git clone https://github.com/user/project.git
```

3. git add

Purpose: Adds new or changed files to the **staging area**, preparing them for the next commit.

Syntax:

```
git add <file-name>
```

Example:

```
git add data.py
```

To add all changes in the current directory:

```
git add .
```

4. git commit

Purpose: Saves the staged changes as a new **commit** in the repository's history. A commit normally includes a message describing the changes.

Syntax:

```
git commit -m "message"
```

Example:

```
git commit -m "Added data analysis code"
```

5. git status

Purpose: Shows the current state of the working directory and staging area. It helps you see which files are **new, modified, staged, or not yet tracked**.

Syntax:

```
git status
```

Example:

```
git status
```

6. git log

Purpose: Displays the **commit history** of the repository, allowing you to see previous commits and their details.

Syntax:

```
git log
```

Example:

```
git log
```

For a shorter history:

```
git log --oneline
```

Simple Git Workflow

Create repository → git init

↓

Check files → git status

↓

Add changes → git add

↓

Save changes → git commit

↓

View history → git log

In simple words:

git init → Start Git
git clone → Copy a repository
git add → Prepare changes
git commit → Save changes
git status → Check changes
git log → View saved history

Working with Remote Repositories on GitHub

A **remote repository** is a repository stored on a server such as GitHub rather than on your computer. It allows you to store your project online and share or synchronize your work with others. The remote repository is commonly given the name **origin**.

1. Connecting a Local Repository to GitHub

After creating a repository on GitHub, you can connect your local Git repository to it using:

```
git remote add origin <repository-URL>
```

To check the remote repository:

```
git remote -v
```

Here, **origin** is the name given to the remote repository.

2. git push

Purpose: Uploads your local commits to the remote repository on GitHub.

Syntax:

```
git push <remote-name> <branch-name>
```

Example:

```
git push origin main
```

This sends the commits from your local main branch to the main branch on GitHub.

In simple words:
Push = Local computer → GitHub

3. git fetch

Purpose: Downloads information about new changes from the remote repository without automatically applying those changes to your current branch.

Syntax:

```
git fetch origin
```

After fetching, you can examine the changes before deciding whether to merge them.

In simple words:
Fetch = Get information about changes from GitHub, but don't apply them yet.

4. git pull

Purpose: Gets the latest changes from the remote repository and integrates them into your current local branch.

git pull essentially performs **fetch + merge**.

Syntax:

```
git pull origin main
```

In simple words:
Pull = Get changes from GitHub and apply them to your local project.

5. git merge

Purpose: Combines the changes from one branch into another branch. It is useful when different people or branches have worked on different parts of a project.

Example:

git merge feature

This merges the feature branch into the **currently active branch**.

If two branches contain conflicting changes, Git may report a **merge conflict**, which must be resolved manually.

In simple words:
Merge = Combine changes from different branches.

Understanding .gitignore Files

A **.gitignore** file tells Git which files and directories should **not be tracked** or included in commits. It is normally placed in the project's root directory.

For example, a Data Science project may contain temporary files, generated output, or environment-specific files that should not be stored in the repository.

A .gitignore file can contain entries such as:

__pycache__/

.venv/

*.log

This tells Git to ignore Python cache files, a virtual environment folder, and log files.

Important: .gitignore mainly affects **untracked files**. A file that has already been tracked by Git is not automatically removed from tracking just because it is later added to .gitignore.

In simple words:
.gitignore = Tell Git which files to ignore.

Creating and Managing Branches

A **branch** is a separate version of the project where you can work on changes without directly affecting the main branch. Branches are commonly used for developing new features, fixing bugs, or experimenting with changes.

Create a branch

```
git branch feature
```

This creates a new branch named feature.

View branches

```
git branch
```

This displays the local branches in the repository.

Switch to a branch

```
git switch feature
```

This switches from the current branch to the feature branch. Git

Create and switch to a new branch

```
git switch -c feature
```

This creates a new branch and immediately switches to it. Git

Delete a branch

git branch -d feature

This deletes the local feature branch when it is no longer needed.

Simple GitHub Workflow

Local Repository → Commit → Push → GitHub

GitHub → Fetch/Pull → Local Repository

Branch → Make Changes → Commit → Merge → Main Branch

In simple words, **remote repositories allow you to keep your project on GitHub, push sends your work to GitHub, fetch gets information about remote changes, pull gets and applies those changes, merge combines branches, .gitignore excludes unwanted files, and branches allow you to work on different versions of a project separately.**